Anexos de evidências

Convívium Management System - Entrega 2, 2026/2. Versão entrega-2-2026-2-revisao-1.

Matriz disciplinar, memória de cálculo, casos/resultados, recorte da arquitetura, registros de integração, capturas e proposta de negócio. Os dados de ensaio são sintéticos.

Matriz final de evidências - Entrega 2

Revisão de 04/11/2026; referência `entrega-2-2026-2`. Os caminhos são relativos à raiz do repositório na tag de referência.

Três disciplinas obrigatórias

Disciplina	Conceito	Aplicação necessária	Localização no código	Teste e evidência
Linguagem de Programação	Tipos, módulos, funções e retorno	Dados do carrinho transformados em solicitação	`frontend/src/lib/portalCliente.js`: `montarSolicitacao`, `totalCarrinho`	`frontend/test/portalCliente.test.js`: subtotal, normalização e entrada inválida
Linguagem de Programação	Entrada/saída, eventos e estado	Consumidor escolhe unidade/itens; equipe recebe a solicitação	`frontend/src/pages/PortalCliente.jsx`; `frontend/src/pages/SolicitacoesMobile.jsx`	API/Auth locais reais aprovados; escrita integral pela interface ainda não homologada; cenários RPC em `operacaoBanco.test.js` aprovados
Linguagem de Programação	Erros e operações assíncronas	Preservar envio pendente quando a resposta de rede é incerta	`frontend/src/lib/pedidoTransacional.js`: `criarEnvioTransacional`	`frontend/test/pedidoTransacional.test.js`: recuperação e cliques simultâneos
Algoritmos e Programação Estruturada	Sequência e seleção	Validar conta, unidade, modalidade, preços, CPF, cotação e itens antes da persistência	`supabase/migrations/20261025004910_empresas_acesso_frete_cliente.sql`: `erp_privado.enviar_mobile`	`frontend/test/operacaoBanco.test.js`: preço adulterado, cotação vencida e titular divergente

Disciplina	Conceito	Aplicação necessária	Localização no código	Teste e evidência
Algoritmos e Programação Estruturada	Repetição e decomposição	Somar linhas/adicionais e processar itens da venda	`frontend/src/lib/portalCliente.js`: `totalCarrinho`; RPCs de envio/aceite	`portalCliente.test.js` e `operacaoBanco.test.js`: adicionais, quantidade e baixa
Algoritmos e Programação Estruturada	Estado, transação e idempotência	Solicitação aguarda sem baixar estoque; aceite cria uma venda; reenvio recupera a mesma operação	`erp_privado.decidir_mobile`, `criar_pedido_transacional`; migrações de portal/empresas e estoque	`operacaoBanco.test.js`: aceite repetido, rollback por falta de estoque e baixa uma vez
Matemática Computacional	Somatório e aritmética monetária	Subtotal com quantidades e adicionais, total e arredondamento	`frontend/src/lib/portalCliente.js`: `totalCarrinho`; `frontend/src/lib/pedidoTransacional.js`: `calcularPrecoCanal`	`portalCliente.test.js`; `operacaoBanco.test.js`: markup coincide com `round(numeric)`
Matemática Computacional	Proporcionalidade e unidades	Taxa de entrega a partir de metros e centavos/km	`frontend/src/lib/entrega.js`: `calcularTaxaEntrega`; função de entrega no servidor	`frontend/test/entrega.test.js`: 3,5 km × R$ 1,50 = R$ 5,25; entradas inválidas
Matemática Computacional	Razões, percentuais e médias	CMV, margem, cobertura de custo e ticket com recorte explícito	`frontend/src/lib/dashboard.js`: `calcularDashboard`	`frontend/test/dashboard.test.js`: CMV 31,90%, cobertura 50%, cancelados e frete separado
Matemática Computacional	Soma ponderada e módulo	Validar dígitos do CPF	`frontend/src/lib/format.js`: `validarCPF`; validação adicional na RPC móvel	`portalCliente.test.js` e `operacaoBanco.test.js`: CPF inválido recusado

A matriz localiza os trechos interpretados nas seções 6 a 8 do trabalho escrito, relacionando conceito, implementação e resultado.

Disciplinas complementares da continuidade

Eixo da Entrega 1	Evidência atual
Desenvolvimento em JavaScript	Componentes React, módulos de domínio, chamadas assíncronas e testes Node.js
Computação em Nuvem	Supabase Auth/PostgREST, PostgreSQL, Edge Function e secrets no servidor; hospedagem externa pendente
Análise Orientada a Objetos	Responsabilidades e relações de Empresa, Loja, Cliente, Pedido, Item, Produto e Variação; o frontend atual usa componentes/funções, não a classe inicial isolada do ZIP

O docente aceitou a linguagem, conforme confirmação do responsável. Os três eixos obrigatórios continuam explicitamente demonstrados. As fontes efetivamente usadas estão na seção 11 do trabalho; a lista institucional completa não foi fornecida no enunciado.

Memória de cálculo implementada - Entrega 2

Referência: `entrega-2-2026-2`; revisão de 04/11/2026. Valores de exemplo são sintéticos. As fórmulas descrevem a implementação atual, substituindo para a Entrega 2 a base matemática do exemplo inicial isolado.

1. Quantidades, preços e subtotal

Para cada linha i: q_i é quantidade inteira positiva; p_i é preço unitário em reais; a_{ij} é preço de cada adicional por unidade. O subtotal é:

$$S = \Sigma_i \left[q_i \times \left(p_i + \Sigma_j a_{ij} \right) \right]$$
$$T = \text{arredondar}(S + F, 2)$$

S, T e F são valores em reais; F é a taxa da cotação válida no delivery próprio, zero em atendimento sem esse frete. O portal soma preços em centavos inteiros antes de converter o resultado para reais. O servidor verifica preços e total; o navegador não determina a taxa cobrada.

Exemplo da validação de banco: item R$ 25,00 + adicional R$ 3,00 = subtotal R$ 28,00; frete R$ 3,75 resulta em R$ 31,75. A quantidade multiplica também os adicionais da linha.

Fontes: `portalCliente.js/totalCarrinho`, `entrega.js/totalComFrete`, RPC de pedido e `operacaoBanco.test.js`. Desconto e cupom não fazem parte deste cálculo atual; o exemplo de desconto do código inicial da Entrega 1 pertence ao histórico.

2. Frete por distância

Variáveis: d é distância de ida por rota em km; m é distância em metros; v é tarifa em R$/km; c é tarifa em centavos/km; f é taxa em centavos.

$$m = \text{arredondar}(d \times 1000)$$
$$c = \text{arredondar}(v \times 100)$$
$$f = \text{arredondar}(m \times c / 1000)$$
$$F = f / 100$$

Entrada	Desenvolvimento	Esperado e obtido no teste
d = 3,5 km; v = R$ 1,50/km	3500 m × 150 centavos/km ÷ 1000 = 525 centavos	R$ 5,25
d = 1,235 km; v = R$ 1,50/km	1235 × 150 ÷ 1000 = 185,25 centavos; arredondar	R$ 1,85
d = 0,01 km; v = R$ 1,50/km	10 × 150 ÷ 1000 = 1,5 centavo; arredondar	R$ 0,02

Fonte: função `calcularTaxaEntrega` em `frontend/src/lib/entrega.js`; teste “taxa proporcional por km, vírgula decimal e arredondamento final” em `entrega.test.js`. O servidor utiliza distância da rota, com cobertura e cotação verificadas; a simulação matemática não comprova a precisão de um endereço.

Entradas vazias, não numéricas, não positivas, além dos limites e abaixo da precisão mínima são recusadas. Não há taxa mínima comercial, cobrança de retorno nem trânsito em tempo real no modelo atual.

3. Markup e arredondamento monetário

p é preço em reais; u é markup percentual. Regra conceitual:

```
preço do canal = arredondar(p × (1 + u/100), 2)
24,90 × 1,15 = 28,635 → R$ 28,64
```

`calcularPrecoCanal` usa inteiros BigInt para centavos e pontos percentuais; o arredondamento coincide com `round(numeric)` do PostgreSQL nos casos testados. A versão anterior podia apresentar R$ 28,63 pela representação decimal em ponto flutuante. Teste de banco: “markup no navegador coincide com round numeric do PostgreSQL”.

4. Custo de ficha e CMV gerencial

Cada ingrediente tem quantidade na unidade cadastrada (g, ml ou un) e custo por essa unidade. Multiplicar quantidade pelo custo unitário e somar compõe o custo de ficha da variação. Não multiplicar gramas por custo em quilogramas sem conversão prévia.

No dashboard:

- C é a soma do custo atual das fichas conhecidas multiplicado pelas quantidades vendidas.

- Rf é a receita dos itens com ficha conhecida.

- Ri é a receita de todos os itens.

- P é o total pago nos pedidos não cancelados; Fe é o frete desses pedidos.

- N é a quantidade desses pedidos.

```
CMV (%) = 100 × C / Rf, se Rf > 0
Margem simplificada (%) = 100 − CMV
Cobertura de custo (%) = 100 × Rf / Ri, se Ri > 0
Faturamento dos itens = P − Fe
Ticket médio (R$/pedido) = P / N, se N > 0
```

Frete não entra na base Rf de CMV. Cancelados não entram nos indicadores. Denominador inválido resulta em indicador indisponível, não margem de 100%.

Caso reproduzido

Pedido não cancelado com dois itens: A por R$ 27,90 com ficha R$ 8,90; B por R$ 27,90 sem ficha conhecida.

Variável ou resultado	Valor
Receita de todos os itens Ri	R$ 55,80
Receita elegível Rf	R$ 27,90

Variável ou resultado	Valor
Custo conhecido C	R$ 8,90
CMV = 8,90 ÷ 27,90 × 100	31,90%
Margem simplificada	68,10%
Cobertura = 27,90 ÷ 55,80 × 100	50%

Esperado e obtido coincidem no teste “calcula CMV agregado apenas sobre itens com ficha e informa cobertura”, em `frontend/test/dashboard.test.js`.

O custo do dashboard é estimado pela ficha atual, podendo mudar após alteração de insumos/fichas. A fotografia de consumo/custo usada no cancelamento é outra evidência operacional. Margem simplificada não é lucro líquido, não inclui todos os gastos e não transforma custo desconhecido em zero.

5. CPF

`validarCPF` normaliza o texto, recusa sequências repetidas e verifica dois dígitos por soma ponderada e resto da divisão por 11. Há validação também no servidor. O identificador sintético 52998224725 é usado em testes; 11111111111 é recusado. Testes não comprovam identidade civil de uma pessoa.

6. Interpretação para a Entrega 2

O modelo satisfaz uma necessidade da aplicação: determinar total/frete e apoiar leitura de custo e venda. Sua validação combina cálculo manual e asserções automatizadas. O relatório e os slides devem declarar as unidades, a base dos percentuais, a regra de arredondamento e as limitações acima.

A fórmula histórica de CMV sobre total com frete, existente no pacote da Entrega 1, difere da regra atual do dashboard, que separa o frete e informa cobertura.

Casos de teste e resultados - Entrega 2

Execução disponível

Campo	Registro
Código funcional	tag `entrega-2-2026-2-revisao-1`; código integrado em `5181371`
Execução local	11/11/2026, Windows, Node.js, diretório `frontend/`
Comando	`npm test` → `node --test`
Obtido	117 testes; 117 aprovados; 0 falhas, cancelados ou ignorados
Banco	36 cenários PostgreSQL/PGlite isolados, com rollback
CI	Execução 43116523681 (https://github.com/terpsandlups/BRASA_BURGER_CRUD_ERP/actions/runs/43116523681), 11/11/2026, três jobs com conclusão success
Passos do CI	Instalação pelo lockfile, testes, build e auditoria de dependências
Limite	Não comprova carga, e-mail externo ou trajetos físicos

A tag original conserva o registro de 04/11; esta tabela identifica a revisão de 11/11 e suas novas verificações. Os testes não acessam o Supabase operacional. O novo teste de bootstrap usa a sequência integral, sem a fixture operacional. A API/Auth foi verificada separadamente na stack local do runner. Os caminhos abaixo são relativos à raiz do repositório; os nomes localizam as asserções e os dados completos. A tabela é um recorte da suíte.

Casos automatizados executados

Caso / entrada	Esperado	Obtido	Interpretação e localização
CT01 - 3,5 km e tarifa 1,50	R$ 5,25	R$ 5,25; aprovado	Proporcionalidade e vírgula decimal; `entrega.test.js`, “taxa proporcional por km...”
CT02 - 1,235 km e tarifa 1,50	R$ 1,85	R$ 1,85; aprovado	Arredondamento final ao centavo; mesmo teste
CT03 - distância/tarifa vazia, zero, negativa ou não numérica	Rejeitar sem taxa	Exceções esperadas; aprovado	Entrada inválida; `entrega.test.js`, “não cobra usando entradas...”
CT04 - preço 24,90 e markup 15%	R$ 28,64 no JS e PostgreSQL	Valores coincidem; aprovado	Regressão de arredondamento; `operacaoBanco.test.js`, “markup no navegador coincide...”
CT05 - duas receitas de 27,90, custo 8,90 conhecido em uma	CMV 31,90%; margem 68,10%; cobertura 50%	Asserções aprovadas	Custo ausente não vira zero; `dashboard.test.js`, “calcula CMV agregado...”
CT06 - pedido cancelado de 99,00	Receita/quantidade zero e ticket indisponível	0, 0 e null; aprovado	Exclusão de cancelados; `dashboard.test.js`, “exclui cancelamentos...”
CT07 - total 33,15 e frete 5,25, mais pedido cancelado	Faturamento 33,15; itens 27,90; frete 5,25	Asserções aprovadas	Separação do frete; `dashboard.test.js`, “separa frete dos itens...”
CT08 - cotação 3,75 enviada e aceita após expiração inicial	Reserva mantém taxa; reenvio recupera solicitação; outro envio não reutiliza cotação	Asserções aprovadas; taxa 3,75	Reserva e idempotência; `operacaoBanco.test.js`, “mobile: reserva é idempotente...”

Caso / entrada	Esperado	Obtido	Interpretação e localização
CT09 - cotação vencida ou de outro titular	Recusar e não criar solicitação	Erros esperados e contagem zero; aprovado	Validade/titular no servidor; `operacaoBanco.test.js`
CT10 - cadastro Auth público	Não criar colaborador nem expor clientes/custos	Asserções aprovadas	Consumidor separado do ERP; `operacaoBanco.test.js`, “mobile: cadastro público...”
CT11 - preço adulterado, adicional repetido, CPF inválido ou quantidade fracionária	Recusar envio	Exceções esperadas; aprovado	Validação no servidor; `operacaoBanco.test.js`, “mobile: preço adulterado...”
CT12 - aceite com estoque insuficiente ou preço atualizado	Reverter venda e conservar solicitação aguardando	Asserções aprovadas	Atomicidade; `operacaoBanco.test.js`, “mobile: estoque insuficiente...”
CT13 - aceite repetido	Mesma venda e baixa única	Asserções aprovadas	Idempotência; `operacaoBanco.test.js`, “mobile: aceite transacional...”
CT14 - administrador e gerente em unidades distintas	Administrador vê suas unidades; gerente apenas a própria e não provisiona conta	Asserções aprovadas	Escopo no banco isolado; `operacaoBanco.test.js`, “empresas: administrador da marca...”
CT15 - mesmo CPF em duas empresas	Cadastros independentes	Asserções aprovadas	Isolamento entre empresas; `operacaoBanco.test.js`, “empresas: master cadastra...”
CT16 - resposta perdida e recarga	Recuperar mesma chave/conteúdo sem duplicação	Asserções aprovadas	Recuperação; `pedidoTransacional.test.js`
CT17 - reposição repetida	Somar saldo atual uma vez	Asserções aprovadas	Reposição atômica; `operacaoBanco.test.js`, “banco: reposição soma...”

Caso / entrada	Esperado	Obtido	Interpretação e localização
CT18 - usuário de leitura, inativo ou gerente de outra unidade	Negar alteração não autorizada	Asserções aprovadas	Autorização/RLS; `operacaoBanco.test.js`

Arquivos: `frontend/test/entrega.test.js`, `dashboard.test.js`, `operacaoBanco.test.js` e `pedidoTransacional.test.js`. A suíte SQL carrega `frontend/test/fixtures/erp.sql` e as migrações indicadas no seu bloco de preparação.

Homologações e alcance da evidência

Caso	Entrada	Esperado	Obtido / interpretação
CT19 / HM01	Sequência integral em dois bancos vazios	Objetos, permissões, primeiro master e dados mínimos	PGlite e PostgreSQL nativo aprovados; também Supabase local real
HM02	Auth real, montagem/envio do frontend e PostgREST local	Solicitação na unidade correta, espera sem baixa, aceite e acompanhamento	R$ 55,80, saldo 100 antes e 98 após aceite; etapa recebido. API aprovada; escrita integral pela interface não realizada
HM03	Duas conexões PostgreSQL e duas sessões Auth simultâneas	Venda e baixa únicas	Aprovado em ambos os ensaios. Nativo: R$ 25,00/150 g; API: R$ 55,80/2 un. Cenários distintos
HM04	Gerente de outra unidade e consumidores	Negar leitura/alteração alheia	Aprovado em SQL e API; isolamento entre empresas adicionalmente coberto por PGlite
HM05	Cancelar com devolução e estoque insuficiente	Repor uma vez, preservar histórico e reverter falha	Nativo: 850 g para 1000 g; API: 98 para 100. Rollback insuficiente aprovado no nativo e suíte SQL

Caso	Entrada	Esperado	Obtido / interpretação
HM06	Endereços reais e rotas físicas	Distância/taxa compatíveis	Não executado em campo. Fórmula e falhas de cobertura testadas
HM07	Origem HTTPS e rota direta	Abrir catálogo e cadastro CPF	Publicado no GitHub Pages e conferido no navegador; CPF dispensa envio de e-mail
HM08	Catálogo e carrinho na interface	Carregar unidade/itens e somar preços	Aprovado em 04/11: 2 x 26,90 = 53,80. Capturas não mostram envio

Resultados sanitizados: `resultado-postgres.json` e `resultado-api-local.json`. A stack Auth/API foi criada e destruída no runner isolado. Nenhuma conta ou pedido sintético foi criado no Supabase compartilhado.

Falhas corrigidas e limites

- O centavo divergente de markup foi corrigido e tem regressão em CT04.

- Preço adulterado, cotação inválida, estoque insuficiente e acesso alheio são rejeições esperadas nos testes.

- O registro anterior relata falha de geocodificação do endereço comercial do Retiro: envio bloqueado e mapa oferecido, sem taxa inventada. Isso comprova tratamento daquela falha, não sucesso de rota do imóvel.

- Respostas simuladas do provedor não demonstram cobertura cartográfica real.

- Carga permanece não testada. Concorrência com conexões independentes foi aprovada; ela não representa carga de produção.

- Auditoria de dependências aprovada não comprova ausência de segredos nem ausência de todo risco.

Modelo de registro para cada execução nova

```
ID do caso e tarefa:
Data e responsável:
Commit/tag e ambiente/versões:
Perfis, empresa e unidade (dados sintéticos):
Entrada e pré-condições:
Resultado esperado:
Resultado obtido:
Interpretação: aprovado, reprovado ou bloqueado; justificativa.
Evidência: arquivo, captura, log sanitizado, consulta ou URL.
Limites e ação decorrente:
```

Revisão para a aula, 11/11/2026

Cadastro direto CPF/senha com limite de rede, senha validada pelo Auth, identificador interno sem e-mail pessoal, CPF imutável no perfil, resolução de conta restrita ao servidor e negação de dados de outra unidade. O aviso Realtime de status chegou ao titular no ensaio real. Consulta ViaCEP e tratamento de falhas foram verificados. Frontend e apresentação publicados por GitHub Actions; capturas HTTPS não criaram contas ou pedidos.

CI integrado: https://github.com/terpsandlups/BRASA_BURGER_CRUD_ERP/actions/runs/43116523681 . Deploy: https://github.com/terpsandlups/BRASA_BURGER_CRUD_ERP/actions/runs/43114129902 .

Arquitetura, fluxo e relações

Representação textual da implementação. As fontes Mermaid editáveis acompanham o ZIP em ARQUITETURA_E_FLUXO.md. Fonte: código do projeto, 2026.

Camada	Fluxo e responsabilidade
Consumidor e equipe	Navegador React acessa Auth e API. Consumidor usa portal da marca; equipe usa ERP
Identidade e acesso	Auth identifica conta; RLS e funções restringem empresa/unidade e titular
Persistência	PostgREST/RPC executa função no PostgreSQL e retorna resultado autorizado
Frete	Edge Function verifica sessão, consulta rota HeiGIT e registra cotação válida
Pedido	Solicitação aguardando sem baixa; aceite transacional gera venda e consumo uma vez
Histórico	Status, cancelamento e destino do consumo preservam rastreabilidade

Relações principais

Empresa possui lojas, catálogo, clientes e usuários ERP. Loja recebe solicitações e pedidos. Pedido contém itens; cada item identifica uma variação do produto. A ficha técnica relaciona variação e ingrediente. Estoque associa ingrediente a loja. Conta Auth envia solicitações próprias, sem implicar perfil de equipe.

Decisões do fluxo

Escolher unidade/modalidade, montar carrinho, autenticar e validar. Se delivery, exigir cotação. Se dados inválidos, corrigir antes de enviar. Se a loja recusar, registrar motivo. Se aceitar, conferir preços e estoque na transação. Se falhar, reverter. Se repetir o aceite, recuperar a mesma venda. Depois, acompanhar status e consultar histórico.

Registro: resultado-postgres.json

Data: 2026-10-29. Ambiente: PostgreSQL nativo local, dois bancos vazios e conexões independentes; contrato Auth de teste.

- Instalação integral em banco vazio 1: aprovada
- Duas conexões de consumidor enviam a mesma chave: uma solicitação e nenhuma baixa
- Duas conexões da equipe aceitam simultaneamente: uma venda de R$ 25,00 e uma baixa de 150 g
- Gerente de outra unidade: leitura vazia e alteração negada
- Cancelamento devolve 150 g e preserva pedido cancelado no histórico
- Estoque insuficiente: rollback sem venda/baixa parcial
- Instalação integral em banco vazio 2: aprovada

CI: https://github.com/terpsandlups/BRASA_BURGER_CRUD_ERP/actions/runs/41924749997

Registro: resultado-api-local.json

Data: 2026-11-11. Ambiente: Supabase local descartável no GitHub Actions: Auth, PostgREST e PostgreSQL reais.

- Cadastro público real pelo módulo do portal cria sessão de consumidor sem acesso de equipe

- CPF e senha passam pelo handler do servidor, Auth real e instalação da sessão no módulo do portal

- Auth real local e catálogo via PostgREST: três unidades e um produto sintético

- Funções do frontend enviam solicitação real: subtotal R$ 55,80 e estoque ainda 100

- Duas sessões Auth independentes aceitam: uma venda R$ 55,80, saldo 98 e acompanhamento recebido

- Gerente de outra unidade não lê nem avança o pedido

- Evento Realtime real avisa o consumidor do status em preparo sem liberar leitura do pedido interno

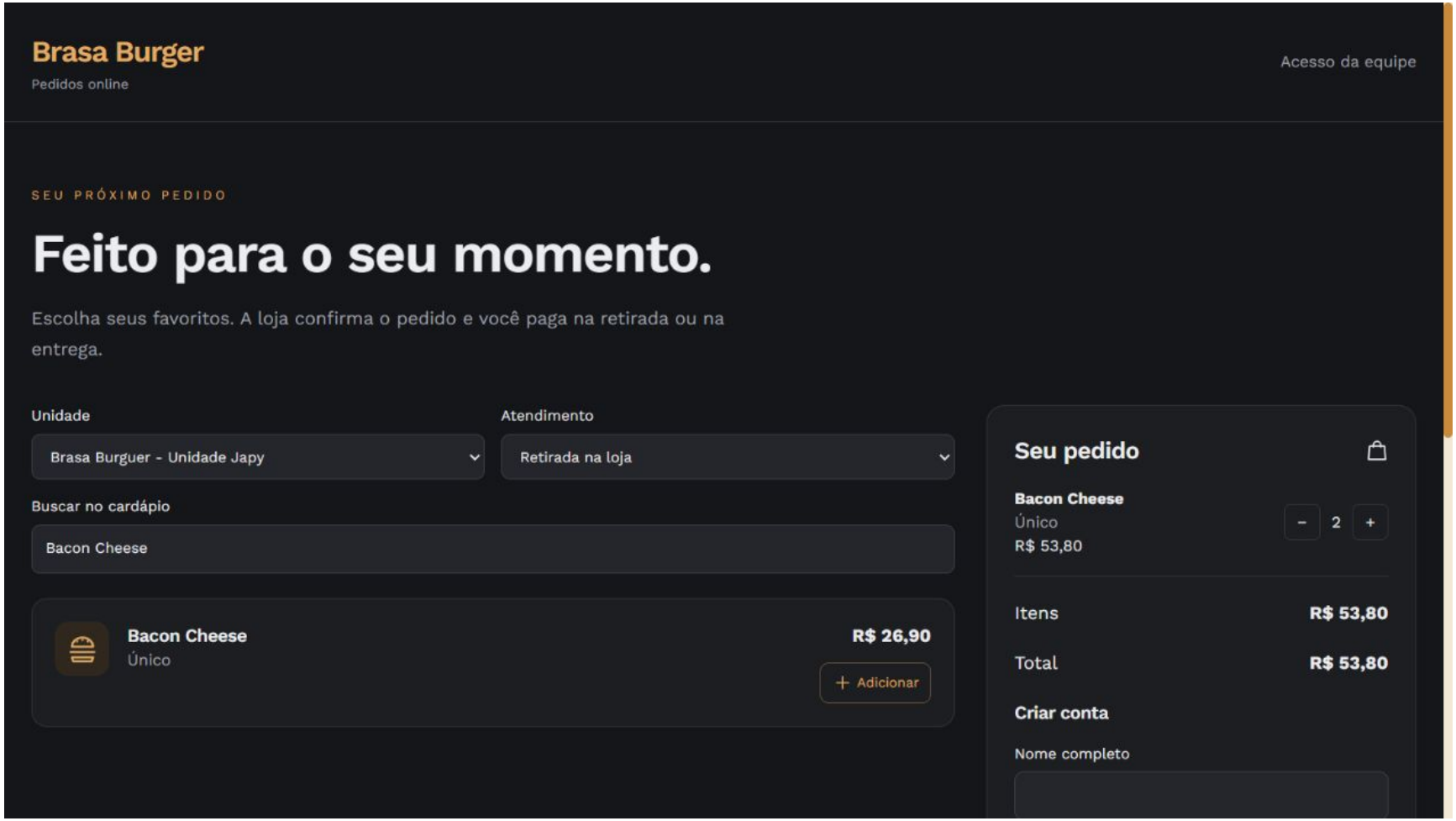
- Cancelamento via API real devolve duas unidades e preserva o histórico

- Aula isolada: 30 cadastros e solicitações concorrentes, 30 avisos Realtime na fila, 30 vendas únicas; estoque 100 na espera, 70 no aceite e 100 após cancelamento

CI: https://github.com/terpsandlups/BRASA_BURGER_CRUD_ERP/actions/runs/43114129801

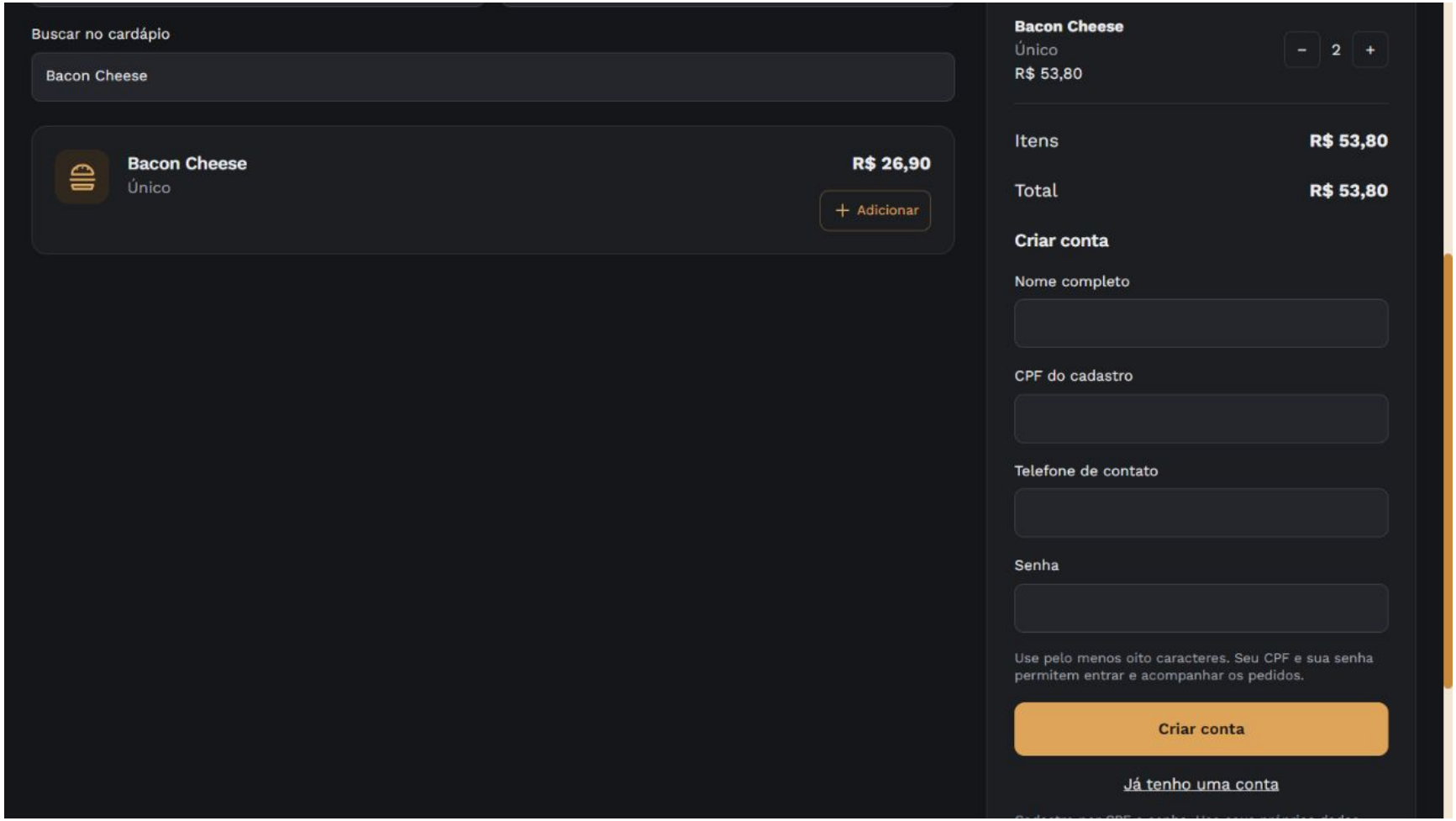
Captura da interface pública

Portal HTTPS da unidade Japy: duas unidades de R$ 26,90 e total R$ 53,80. Sem envio. Conferência em 11/11/2026. Fonte: próprio projeto.



Captura da interface pública

Cadastro com CPF e senha. Campos vazios, sem conta, solicitação ou venda criada nesta captura. Conferência em 11/11/2026. Fonte: próprio projeto.



Modelo de negócio proposto para o Convívium

O Convívium Management System é a plataforma. A empresa cliente usa seu nome, logo e destaque no ERP e no portal. Brasa Burger é o primeiro caso de aplicação. O estabelecimento é o cliente comercial do software; o consumidor compra alimento da loja e acompanha seu próprio pedido.

Oferta e receita

A oferta reúne catálogo, fichas, estoque, pedidos, histórico, indicadores e portal por marca. A proposta comercial considera assinatura por empresa e faixa de unidades. Implantação, migração, treinamento e personalização podem ser serviços opcionais com escopo definido. Preços, planos e cobrança recorrente não estão implementados ou contratados nesta entrega.

A venda de alimentos e o frete pertencem à operação do estabelecimento. Não se presume comissão por pedido, contratação de entregadores ou repasse financeiro pela plataforma. O pagamento atual ocorre diretamente com a loja.

Custos e implantação

Os custos a apurar incluem infraestrutura, serviços de rota/e-mail, manutenção, suporte, implantação e evolução. Não há estimativa financeira validada neste material.

A implantação proposta passa por identidade/unidades, catálogo/fichas/saldos, contas/perfis, conferência de um pedido completo e treinamento. O dono do banco provisiona o primeiro master; o administrador vincula as contas existentes da equipe.

Validação comercial

Medir tempo até o primeiro pedido, uso dos módulos, erros de atendimento, custo de suporte e implantação, aceitação da assinatura e retenção. São métricas propostas, ainda sem valores medidos. Pesquisa aprofundada de concorrentes permanece dependente do conector Search/Exa.

Fontes e estágio

Baseado nos módulos atuais, nas duas apresentações fornecidas no repositório e na apresentação demonstrativa compartilhada: https://brasa-burguer-apresentacao-2026.azurebard4.chatgpt.site/.

A apresentação web contém simulações e não é comprovação de publicação do ERP ou métricas de produção. A plataforma é um piloto funcional. Convites, tema completo, integrações e oferta comercial consolidada são evoluções.